

Documentação de Tecnologias

1. Visão geral

O projeto consiste no desenvolvimento de um MiniCRM de leads para a Sol Nascente Motos, concessionária Honda.

O sistema possui três partes principais:

- Landing Page pública para cadastro de leads;
- API responsável pelo gerenciamento dos dados;
- Painel administrativo para autenticação e acompanhamento dos leads.

A solução foi planejada priorizando simplicidade, velocidade de desenvolvimento, manutenção e facilidade de deploy.

2. Stack tecnológica

2.1 Frontend

React.js + TypeScript + Vite

O frontend será desenvolvido utilizando React.js com TypeScript.

O React será responsável pela construção da interface e pelo gerenciamento dos componentes da aplicação, enquanto o TypeScript será utilizado para fornecer tipagem estática ao código.

O Vite será utilizado como ferramenta de build e ambiente de desenvolvimento.

Justificativa

React é adequado para a construção tanto da Landing Page quanto do painel administrativo. O TypeScript reduz erros relacionados a tipos e facilita a manutenção do projeto.

O Vite proporciona um ambiente de desenvolvimento rápido e uma configuração simples.

3. Interface e responsividade

Tailwind CSS

O Tailwind CSS será utilizado para estilização da aplicação.

Justificativa

O sistema possui poucas telas e precisa funcionar bem principalmente em dispositivos móveis. O Tailwind permite desenvolver rapidamente interfaces responsivas sem a necessidade de criar uma grande quantidade de arquivos CSS.

A responsividade será considerada desde a implementação da Landing Page até o painel administrativo.

4. Gerenciamento e validação de formulários

React Hook Form

O React Hook Form será utilizado para gerenciamento dos formulários.

Zod

O Zod será utilizado para definir e validar os dados recebidos pelos formulários.

Justificativa

O sistema possui um formulário público de cadastro de leads e necessita validar campos como nome, WhatsApp, modelo de interesse e unidade.

A combinação React Hook Form + Zod permite centralizar as regras de validação e manter o código organizado.

Exemplos de validações:

- Nome obrigatório;
 - WhatsApp obrigatório;
 - Modelo de interesse obrigatório;
 - Unidade obrigatória;
 - Formato válido para telefone.
-

5. Backend

Node.js + Express + TypeScript

O backend será desenvolvido utilizando Node.js, Express e TypeScript.

A API será responsável pela comunicação entre o frontend e os serviços de dados.

Responsabilidades da API

- Receber novos leads;
- Validar dados recebidos;

- Salvar leads;
- Listar leads;
- Atualizar o status dos leads;
- Controlar o acesso às operações administrativas.

Justificativa

Express fornece uma estrutura simples para criação de uma API REST e é suficiente para o escopo do MiniCRM.

Não será utilizado NestJS porque o projeto possui poucas funcionalidades e não necessita da estrutura adicional oferecida pelo framework.

6. Banco de dados

Firestore

O Firestore será utilizado como banco de dados do sistema.

A principal coleção será:

leads

Cada documento representará um lead.

Exemplo conceitual:

```
leads
├── leadId
│   ├── name
│   ├── whatsapp
│   ├── model
│   ├── unit
│   ├── status
│   ├── createdAt
│   └── updatedAt
```

Status possíveis

```
NEW
CONTACTED
CONVERTED
LOST
```

Justificativa

O Firestore foi escolhido porque o projeto possui uma estrutura de dados relativamente simples e não necessita de consultas relacionais complexas.

Além disso, o Firebase reduz a quantidade de infraestrutura necessária para o desenvolvimento e permite utilizar autenticação e banco de dados dentro do mesmo ecossistema.

7. Autenticação

Firestore Authentication

O Firebase Authentication será utilizado para o login do painel administrativo.

O sistema terá autenticação por:

- E-mail;
- Senha.

Somente usuários autenticados poderão acessar o painel administrativo e visualizar ou alterar leads.

Justificativa

O desafio exige um login simples para o painel administrativo. Utilizar um serviço especializado evita implementar manualmente funcionalidades como armazenamento seguro de senhas, geração de tokens e gerenciamento básico de autenticação.

8. Comunicação entre frontend e backend

A comunicação será realizada utilizando HTTP e uma API REST.

Exemplo:

```
Frontend
  ↓
HTTP Request
  ↓
Node.js / Express API
  ↓
Firestore
```

Endpoints previstos:

```
POST    /api/leads
GET      /api/leads
PATCH  /api/leads/:id/status
```

O frontend poderá utilizar `fetch` ou `Axios` para realizar as requisições.

Para esse projeto, `fetch` é suficiente e evita adicionar uma dependência desnecessária.

9. Controle de acesso

As rotas administrativas da API deverão exigir autenticação.

O fluxo será:

```
Administrador
  ↓
Login Firebase
  ↓
Token de autenticação
  ↓
Frontend
  ↓
API
  ↓
Validação do token
  ↓
Operação autorizada
```

Dessa forma, o acesso aos dados administrativos não dependerá apenas da proteção visual das páginas do frontend.

10. Deploy

Frontend

O frontend será hospedado na Vercel.

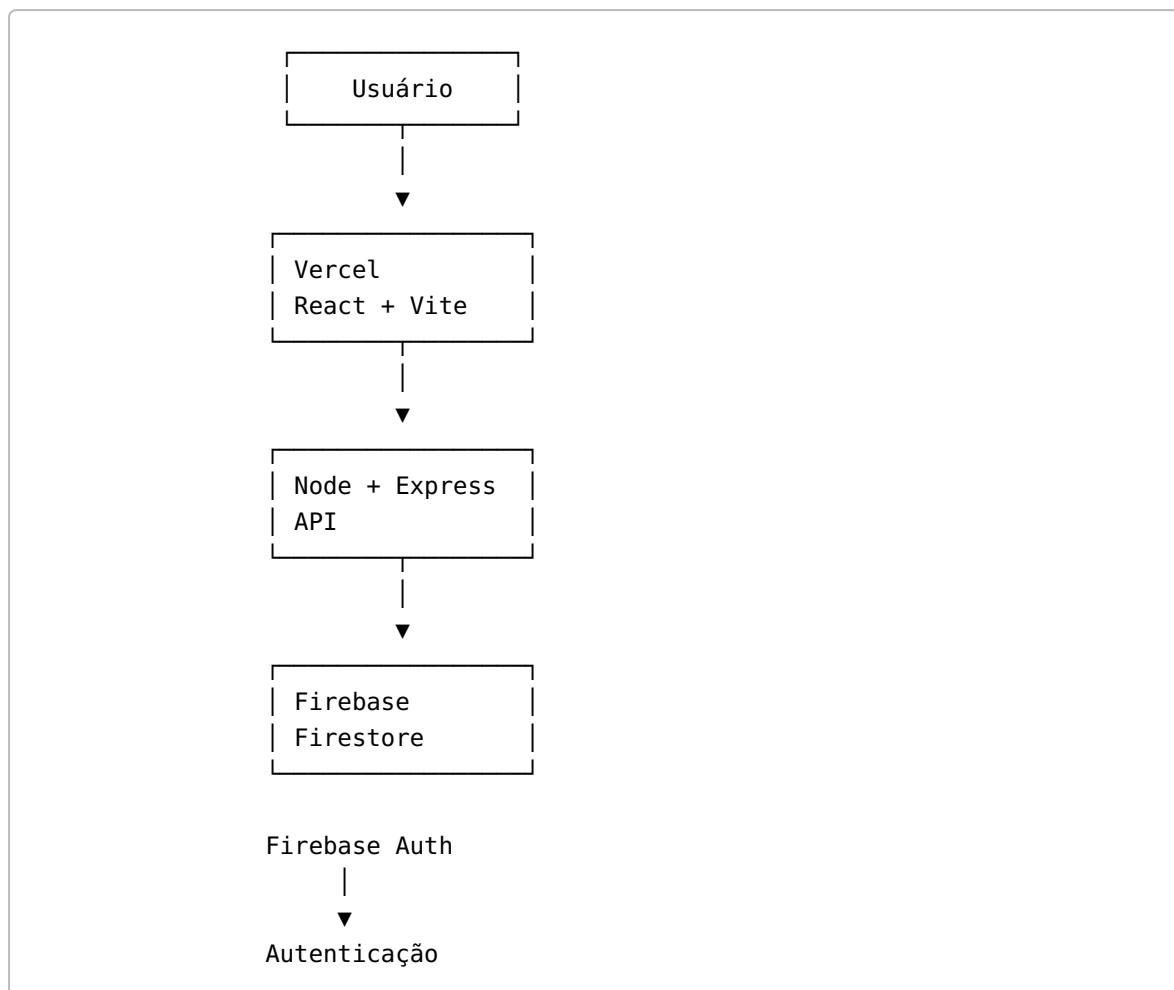
Backend

A API Node.js poderá ser hospedada no Render ou Railway.

Firebase

O Firestore e o Firebase Authentication serão utilizados como serviços gerenciados.

Arquitetura de deploy:



11. Tecnologias que não serão utilizadas

Algumas tecnologias poderiam ser utilizadas, mas foram consideradas desnecessárias para este projeto.

NestJS

Não será utilizado porque o backend possui poucas rotas e regras de negócio.

Redux

Não será utilizado porque o estado global da aplicação é pequeno. React Context ou estado local são suficientes.

Prisma

Não será utilizado porque o banco escolhido é Firestore, que não necessita de ORM relacional.

MongoDB

Não será utilizado porque o Firestore já atende ao escopo e permite utilizar autenticação e banco dentro do mesmo ecossistema.

Docker

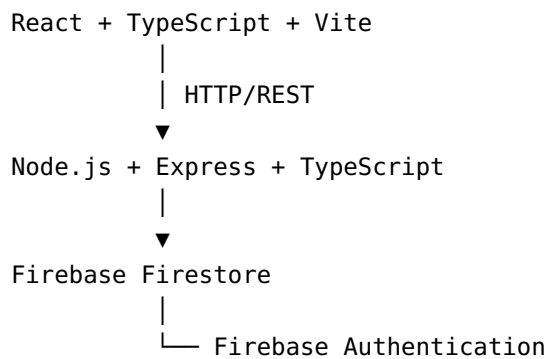
Não será utilizado inicialmente porque adicionaria configuração de infraestrutura sem necessidade para a entrega.

Next.js

Não será utilizado porque o projeto não necessita de SSR, SEO avançado ou funcionalidades específicas do framework.

12. Resumo da decisão arquitetural

A arquitetura foi definida com foco em simplicidade e velocidade de desenvolvimento.



Essa arquitetura atende ao escopo atual sem introduzir complexidade desnecessária.

Caso o sistema evolua para produção, poderão ser adicionados recursos como testes automatizados, observabilidade, controle de permissões, logs, CI/CD, rate limiting e uma arquitetura de backend mais robusta.